

Module 2: Finite State Automata

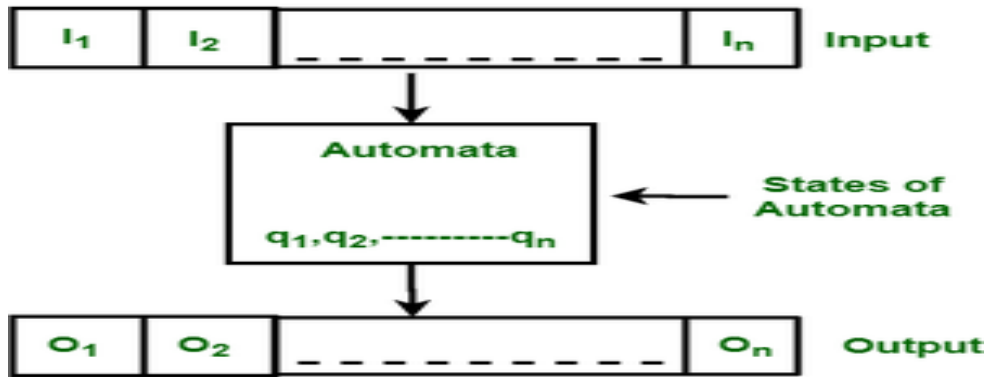
Automata Theory is a branch of the Theory of Computation. It deals with the study of abstract machines and their capacities for computation. An abstract machine is called the automata. It includes the design and analysis of automata, which are mathematical models that can perform computations on strings of symbols according to a set of rules.

Why do we study Theory of Computation?

- **Regular Expressions (RE)** : Used for pattern matching in Linux/Unix command prompt, programming languages and XML/DTD to describe structure.
- **Finite Automata in Modeling Systems** : Used in designing and checking models and electronic circuits that operate based on certain rules.
- **Context-Free Grammars (CFG)** : Used in Compiler / Programming Language design to describe syntax and natural language processing to describe structure.
- **Mathematical Models** : Mathematical understanding of computing devices by mathematically modeling them.
- **Building Block for Quantum Computing**: Turing Machines (we study in this subject) are considered a fundamental building block for understanding quantum computation models.
- **Optimizing Algorithm Efficiency** : Helps classify problems based (e.g., **P**, **NP**, **NP-complete**, and **NP-hard**), proving that some problems have no efficient solutions.
- **Understanding Computability** : Study of which problems can be solved using algorithms, essentially defining the boundaries of what a computer can calculate.. Problems like the "Halting Problem" which are demonstrably impossible to solve with a general algorithm.

Introduction of Finite Automata: Finite automata are abstract machines used to recognize patterns in input sequences, forming the basis for understanding regular languages in computer science.

- Consist of states, transitions, and input symbols, processing each symbol step-by-step.
- If it ends in an accepting state after processing the input, then the input is accepted; otherwise, rejected.
- Finite automata come in deterministic (DFA) and non-deterministic (NFA), both of which can recognize the same set of regular languages.
- Widely used in text processing, compilers, and network protocols.



Features of Finite Automata

- **Input:** Set of symbols or characters provided to the machine.
- **Output:** Accept or reject based on the input pattern.
- **States of Automata:** The conditions or configurations of the machine.
- **State Relation:** The transitions between states.
- **Output Relation:** Based on the final state, the output decision is made.

Formal Definition of Finite Automata

A finite automaton can be defined as a tuple:

$\{ Q, \Sigma, q, F, \delta \}$, where:

- Q : Finite set of states
- Σ : Set of input symbols
- q : Initial state
- F : Set of final states
- δ : Transition function

Types of Finite Automata

There are two types of finite automata:

- Deterministic Finite Automata (DFA)
- Non-Deterministic Finite Automata (NFA)

1. Deterministic Finite Automata (DFA)

A DFA is represented as $\{Q, \Sigma, q, F, \delta\}$. In DFA, for each input symbol, the machine transitions to one and only one state. DFA does not allow any null transitions, meaning every state must have a transition defined for every input symbol.

DFA consists of 5 tuples $\{Q, \Sigma, q, F, \delta\}$.

Q : set of all states.

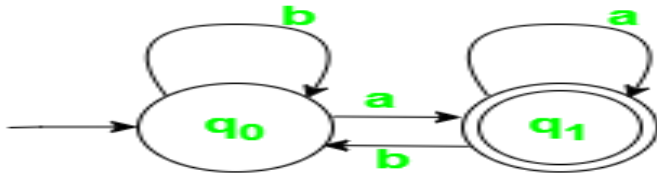
Σ : set of input symbols. (Symbols which machine takes as input)

q : Initial state. (Starting state of a machine)

F : set of final state.

δ : Transition Function, defined as $\delta : Q \times \Sigma \rightarrow Q$.

Example: Construct a DFA that accepts all strings ending with 'a'. Given: $\Sigma = \{a, b\}$, $Q = \{q_0, q_1\}$, $F = \{q_1\}$



State\Symbol	a	b
q0	q1	q0
q1	q1	q0

In this example, if the string ends in 'a', the machine reaches state q_1 , which is an accepting state.

2) Non-Deterministic Finite Automata (NFA)

NFA is similar to DFA but includes the following features:

- It can transition to multiple states for the same input.
- It allows null (ϵ) moves, where the machine can change states without consuming any input.

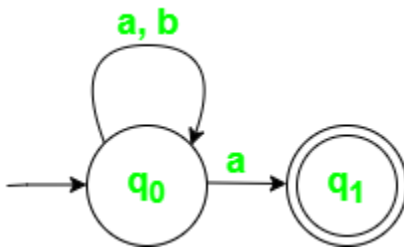
Formal Definition of an NDFA

An NDFA can be represented by a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where –

- **Q** is a finite set of states.
- **Σ** is a finite set of symbols called the alphabets.
- **δ** is the transition function where $\delta: Q \times \Sigma \rightarrow 2^Q$
(Here the power set of Q (2^Q) has been taken because in case of NDFA, from a state, transition can occur to any combination of Q states)
- **q_0** is the initial state from where any input is processed ($q_0 \in Q$).
- **F** is a set of final state/states of Q ($F \subseteq Q$).

Example:

Construct an NFA that accepts strings ending in 'a'. Given: $\Sigma = \{a, b\}$, $Q = \{q_0, q_1\}$, $F = \{q_1\}$



State Transition Table for above Automaton,

State\Symbol	a	b
q_0	$\{q_0, q_1\}$	q_0

q1	ϕ	ϕ
----	--------	--------

In an NFA, if any transition leads to an accepting state, the string is accepted.

Comparison of DFA and NFA

Although NFAs appear more flexible, they do not have more computational power than DFAs. Every NFA can be converted to an equivalent DFA, although the resulting DFA may have more states.

- DFA: Single transition for each input symbol, no null moves.
- NFA: Multiple transitions and null moves allowed.
- Power: Both DFA and NFA recognize the same set of regular languages.

NFA with epsilon transition

An NFA with epsilon transitions (ϵ -NFA) is a Non-deterministic Finite Automaton that allows transitions between states *without consuming any input symbol*, denoted by the Greek letter epsilon (ϵ) or lambda (λ). These "empty moves" provide more flexibility, letting the machine move between possibilities (like guessing the start of a pattern) before reading an input, enabling acceptance of languages like the empty string and simplifying design, though they can be converted to standard NFAs.

ϵ -NFAs add a convenient feature but (in a sense) they bring us nothing new. They do not extend the class of languages that can be represented.

Both NFAs and ϵ -NFAs recognize exactly the same languages.

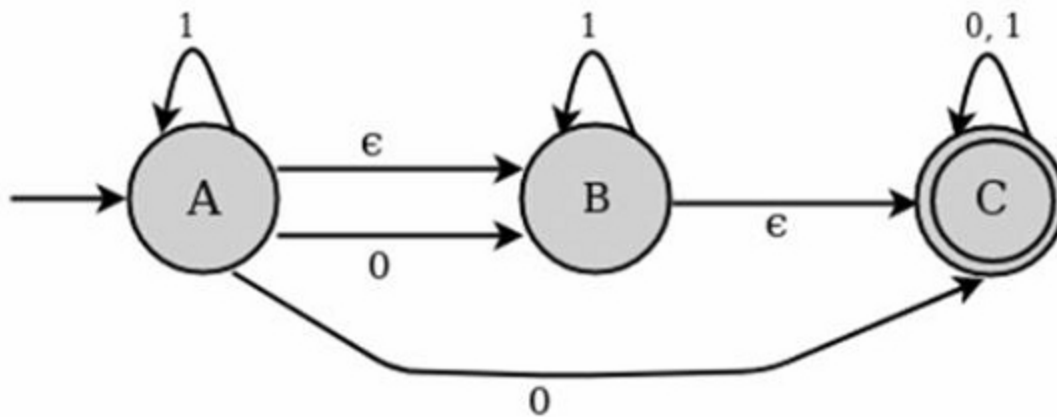
Epsilon (ϵ) - closure

Epsilon closure for a given state X is a set of states which can be reached from the states X with only (null) or E moves including the state X itself.

In other words, ϵ -closure for a state can be obtained by union operation of the ϵ -closure of the states which can be reached from X with a single E move in a recursive manner.

Example

Consider the following figure of NFA with ϵ move –



The transition state table for the above NFA is as follows –

State	0	1	epsilon
A	B,C	A	B
B	-	B	C
C	C	C	-

For the above example, ϵ closure are as follows –

ϵ closure(A) : {A, B,C}
 ϵ closure(B) : {B,C}
 ϵ closure(C) : {C}

Minimization of DFA

DFA minimization stands for converting a given DFA to its equivalent DFA with a minimum number of states. DFA minimization is also called Optimization of DFA and uses partitioning algorithms.

Minimization of DFA

Suppose there is a DFA $D = \langle Q, \Delta, q_0, \Delta, F \rangle$ which recognizes a language L. Then the minimized DFA $D = \langle Q', \Delta, q_0, \Delta, F' \rangle$ can be constructed for language L as:

Step 1: We will divide Q (set of states) into two sets. One set will contain all final states and other set will contain non-final states. This partition is called P_0 .

Step 2: Initialize $k = 1$

Step 3: Find P_k by partitioning the different sets of P_{k-1} . In each set of P_{k-1} , we will take all possible pair of states. If two states of a set are distinguishable, we will split the sets into different sets in P_k .

Step 4: Stop when $P_k = P_{k-1}$ (No change in partition)

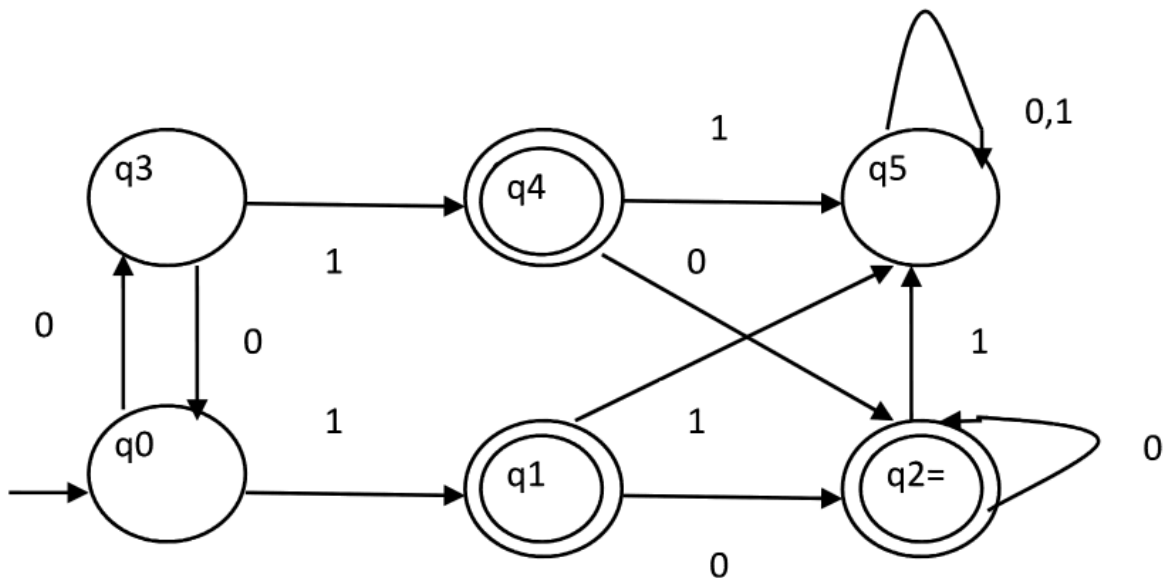
Step 5: All states of one set are merged into one. No. of states in minimized DFA will be equal to no. of sets in P_k .

How to find whether two states in partition P_k are distinguishable ?

Two states (q_i, q_j) are distinguishable in partition P_k if for any input symbol a , $\Delta(q_i, a)$ and $\Delta(q_j, a)$ are in different sets in partition P_{k-1} .

Example

Consider the following DFA shown in figure.



Step 1. P_0 will have two sets of states. One set will contain q_1, q_2, q_4 which are final states of DFA and another set will contain remaining states. So $P_0 = \{ \{ q_1, q_2, q_4 \}, \{ q_0, q_3, q_5 \} \}$.

Step 2. To calculate P1, we will check whether sets of partition P0 can be partitioned or not:

i) For set { q1, q2, q4 } :

$\Delta(q1, 0) = \Delta(q2, 0) = q2$ and $\Delta(q1, 1) = \Delta(q2, 1) = q5$, So q1 and q2 are not distinguishable.

Similarly, $\Delta(q1, 0) = \Delta(q4, 0) = q2$ and $\Delta(q1, 1) = \Delta(q4, 1) = q5$, So q1 and q4 are not distinguishable.

Since, q1 and q2 are not distinguishable and q1 and q4 are also not distinguishable, so q2 and q4 are not distinguishable. So, { q1, q2, q4 } sets will not be partitioned in P1.

ii) For set { q0, q3, q5 } :

$\Delta(q0, 0) = q3$ and $\Delta(q3, 0) = q0$

$\Delta(q0, 1) = q1$ and $\Delta(q3, 1) = q4$

Moves of q0 and q3 on input symbol 0 are q3 and q0 respectively which are in the same set in partition P0. Similarly, Moves of q0 and q3 on input symbol 1 are q1 and q4 which are in the same set in partition P0. So, q0 and q3 are not distinguishable.

$\Delta(q0, 0) = q3$ and $\Delta(q5, 0) = q5$ and $\Delta(q0, 1) = q1$ and $\Delta(q5, 1) = q5$

Moves of q0 and q5 on input symbol 1 are q1 and q5 respectively which are in different set in partition P0. So, q0 and q5 are distinguishable. So, the set { q0, q3, q5 } will be partitioned into { q0, q3 } and { q5 }. So,

$P1 = \{ \{ q1, q2, q4 \}, \{ q0, q3 \}, \{ q5 \} \}$

To calculate P2, we will check whether sets of partition P1 can be partitioned or not:

iii) For set { q1, q2, q4 } :

$\Delta(q1, 0) = \Delta(q2, 0) = q2$ and $\Delta(q1, 1) = \Delta(q2, 1) = q5$, So q1 and q2 are not distinguishable.

Similarly, $\Delta(q1, 0) = \Delta(q4, 0) = q2$ and $\Delta(q1, 1) = \Delta(q4, 1) = q5$, So q1 and q4 are not distinguishable.

Since, q1 and q2 are not distinguishable and q1 and q4 are also not distinguishable, so q2 and q4 are not distinguishable. So, { q1, q2, q4 } sets will not be partitioned in P2.

iv) For set { q0, q3 } :

$$\Delta(q_0, 0) = q_3 \text{ and } \Delta(q_3, 0) = q_0$$

$$\Delta(q_0, 1) = q_1 \text{ and } \Delta(q_3, 1) = q_4$$

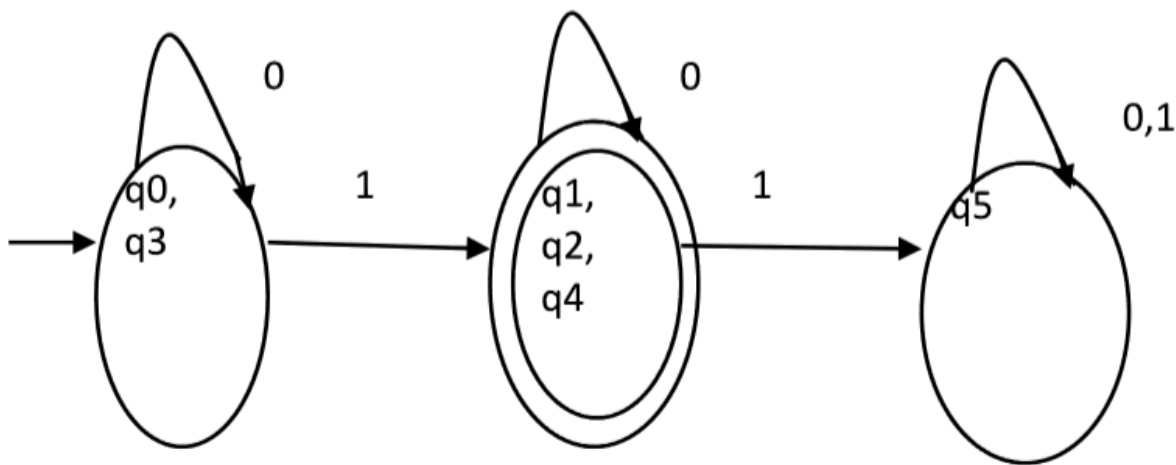
Moves of q0 and q3 on input symbol 0 are q3 and q0 respectively which are in the same set in partition P1. Similarly, Moves of q0 and q3 on input symbol 1 are q1 and q4 which are in the same set in partition P1. So, q0 and q3 are not distinguishable.

v) For set { q5 }:

Since we have only one state in this set, it can't be further partitioned. So,

$$P_2 = \{ \{ q_1, q_2, q_4 \}, \{ q_0, q_3 \}, \{ q_5 \} \}$$

Since, $P_1 = P_2$. So, this is the final partition. Partition P2 means that q1, q2 and q4 states are merged into one. Similarly, q0 and q3 are merged into one. Minimized DFA corresponding to DFA of Figure 1 is shown in Figure 2 as:

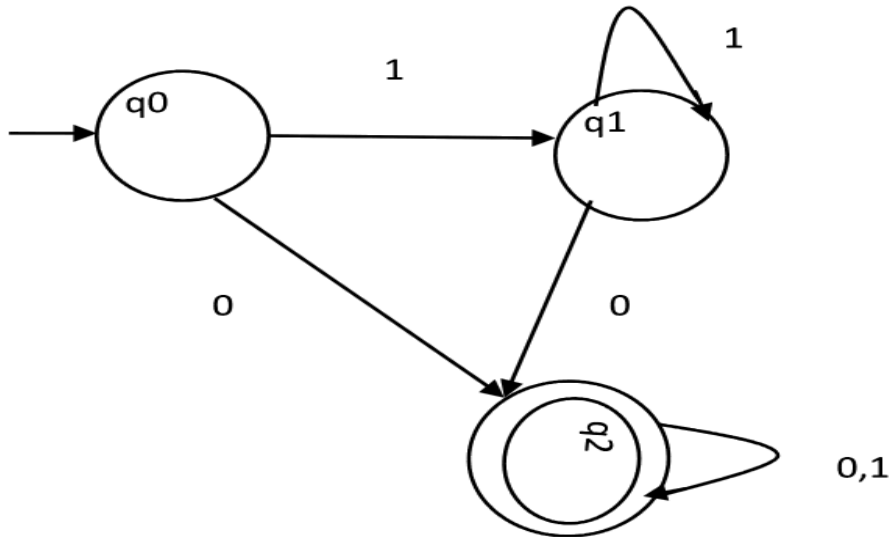


Question : Consider the given DFA. Which of the following is false?

1. Complement of $L(A)$ is context-free.
2. $L(A) = L((11^*0 + 0)(0 + 1)^*0^*1^*)$

3. For the language accepted by A, A is the minimal DFA.

4. A accepts all strings over $\{0, 1\}$ of length at least two.



A. 1 and 3 only

B. 2 and 4 only

C. 2 and 3 only

D. 3 and 4 only

Solution : Statement 4 says, it will accept all strings of length at least 2. But it accepts 0 which is of length 1. So, 4 is false.

Statement 3 says that the DFA is minimal. We will check using the algorithm discussed above.

$P_0 = \{ \{ q_2 \}, \{ q_0, q_1 \} \}$

$P_1 = \{ q_2 \}, \{ q_0, q_1 \}$. Since, $P_0 = P_1$, P_1 is the final DFA. q_0 and q_1 can be merged. So minimal DFA will have two states. Therefore, statement 3 is also false.

So the correct option is (D).

Advantages:

Reduced Complexity: Minimizing a DFA reduces its complexity by eliminating redundant states and transitions. The resulting minimized DFA is typically smaller and simpler, making it easier to understand, analyze, and modify. Minimization improves the efficiency of executing the DFA and reduces the resources required for its implementation.

Optimal Space Utilization: A minimized DFA occupies less memory and storage space compared to the original DFA. By eliminating redundant states, the minimized DFA requires fewer resources to store and represent its states and transitions. This is advantageous when working with large DFAs or limited memory environments.

Improved Performance: Minimizing a DFA can significantly improve its performance in terms of speed and efficiency. With fewer states and transitions, the minimized DFA requires fewer computations and memory accesses, resulting in faster execution. This is particularly beneficial when processing large input strings or handling real-time applications.

Language Equivalence: Minimization ensures that the resulting DFA is language equivalent to the original DFA. That is, it recognizes the same language, accepting the same valid strings and rejecting the same invalid strings. Minimization preserves the language semantics while improving the DFA's structure.

Disadvantages:

Increased Computational Complexity: Minimizing a DFA involves performing state equivalence calculations and merging equivalent states. The computational complexity of the minimization process can be significant, especially for large DFAs. As the DFA's size grows, the time and computational resources required for minimization increase, making it computationally expensive in some cases.

Additional Design and Analysis Effort: Minimization requires additional design and analysis effort compared to working with the original DFA. Determining state equivalence and merging equivalent states often involves careful analysis of the DFA's structure and transition patterns. This process can be time-consuming and error-prone, requiring attention to detail.

Loss of Readability: In some cases, the minimized DFA may become less readable and intuitive compared to the original DFA. The merging of equivalent states may result in complex state representations, making it more challenging to interpret the DFA's behavior or understand its structure.

Limited to Deterministic Automata: Minimization techniques are applicable only to deterministic automata like DFAs. Non-deterministic automata, such as NFAs, cannot be directly minimized. Converting an NFA to a DFA before minimizing it adds an additional step and may lead to an increase in the DFA's size.

Operations on DFA

Deterministic Finite Automata (DFA) are simple machines that help computers recognize patterns in data. They follow a set of fixed rules and read input step by step to determine if it belongs to a specific language. DFAs are widely used in search engines, spell-checkers, and text-processing systems.

They are fundamental computational models used in automata theory to recognize regular languages. Various operations on DFAs allow us to construct new automata from existing ones, enabling us to build complex language recognizers.

One of the most interesting aspects of DFAs is that they can be combined or modified using different operations. Here are some key operations on DFAs, explained with examples:

1. Union of Two DFAs ($L_1 \cup L_2$)

Union means combining two DFAs so that the new DFA accepts all words that either of the original DFAs would accept. Example: Let,

- DFA 1 (L_1) accepts $L_1 = \{a, ab\}$
- DFA 2 (L_2) accepts $L_2 = \{b, ab\}$
- The union DFA accepts $L_1 \cup L_2 = \{a, ab, b\}$

Union Process in DFA

In theory of computation, union of two Deterministic Finite Automata (DFAs) is an operation used to construct a new DFA that recognizes the union of languages accepted by the original automata. For example, two DFAs each recognizing a language L_1 and L_2 respectively, their union DFA will accept any string that belongs to either L_1 or L_2 .

Constructing a DFA for the Union of Two DFAs

To perform the union operation on two deterministic finite automata (DFAs), the following steps can be taken:

Step 1: Create a new DFA with a new set of states, consisting of all the states from both original DFAs.

Step 2: Define the initial state of the new DFA to be the tuple (q_1, q_2) , where q_1 and q_2 are the initial states of the original DFAs.

Step 3: For each state in the new DFA, define the transition function by taking the union of the transition functions of the original DFAs. For example, if the transition function for the first DFA is $\delta_1(q_1, a) = q_2$, and the transition function for the second DFA is $\delta_2(q_3, a) = q_4$, then the transition function for the new DFA is $\delta((q_1, q_3), a) = (q_2, q_4)$.

Step 4: Define the set of final states of the new DFA to be the union of the sets of final states of the original DFAs.

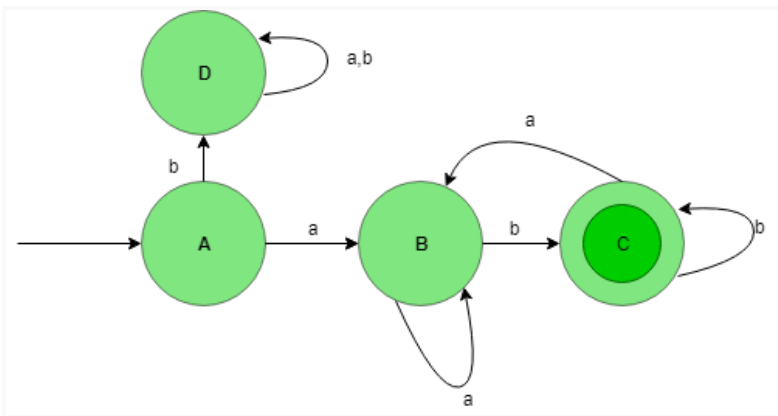
Step 5: The resulting DFA will recognize the language that is the union of the languages recognized by the original DFAs.

Example: Let's understand the Union process in Deterministic Finite Automata (DFA) with the help of the below example. Designing a DFA for the set of strings over $\{a, b\}$ such that the string of the language starts and ends with different symbols. These two desired languages will be formed:

$L1 = \{ab, aab, aabab, \dots\}$
 $L2 = \{ba, bba, bbaba, \dots\}$

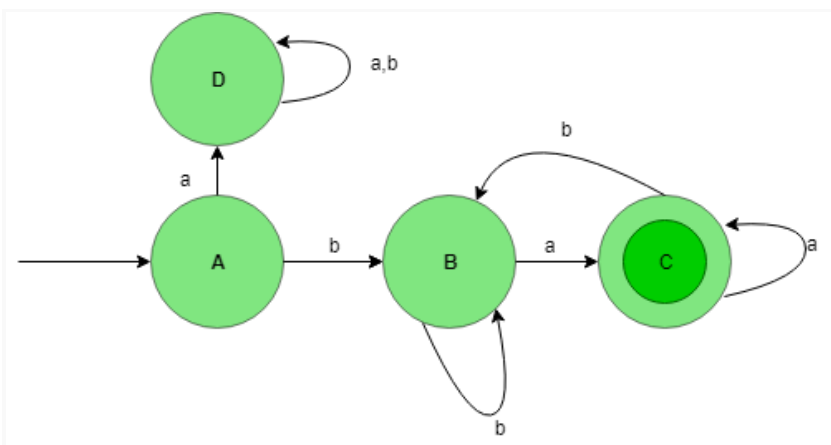
$L1 = \{\text{starts with } a \text{ and ends with } b\}$ and $L2 = \{\text{starts with } b \text{ and ends with } a\}$. Then $L = L1 \cup L2$ or $L = L1 + L2$

State Transition Diagram for the language L1:



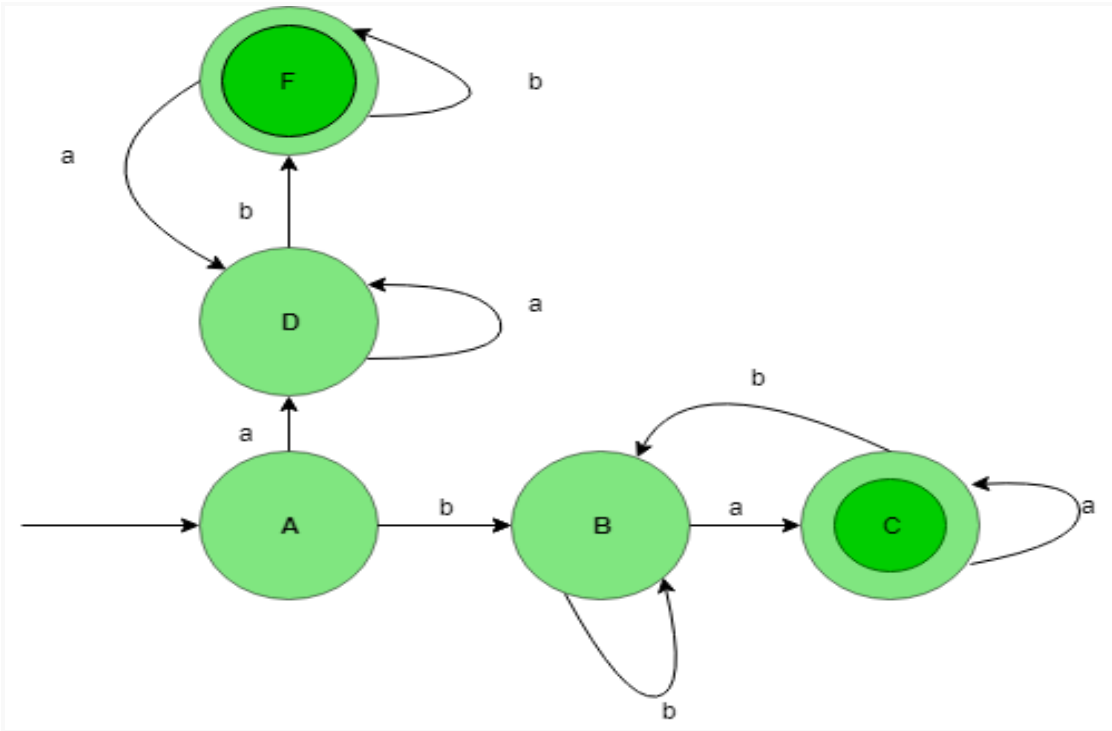
This DFA accepts all the strings starting with a and ending with b . Here, State A is the initial state and state C is the final state.

State Transition Diagram for the language L2:



This DFA accepts all the strings starting with b and ending with a . Here, State A is the initial state and state C is the final state. Now, taking the union of L1 and L2 language gives the final result of the language which starts and ends with different elements.

State Transition Diagram of $L1 \cup L2$:



Thus as we see that L_1 and L_2 have been combined through the union process and this final DFA accepts all the languages containing strings starting and ending with different symbols.

Note: From the above example, we can also infer that regular languages are closed under union(i.e. Union of two regular languages is also regular).

Applications of Union in DFAs

- **Lexical Analysis of Compilers:** Union of DFAs allows for the creation of lexical analyzers that recognize more than one token pattern in one pass.
- **Pattern Matching in Text Processing:** Union operations permit the union of various search patterns into a single automaton to scan text efficiently.
- **Merging Several Language Acceptors:** Union makes it possible to merge individual DFAs to create automata that accept the union of several regular languages.
- **Designing Complicated Input Validators:** Union of DFAs is employed to check inputs that should conform to one out of many format rules.
- **Simplifying Automata-Based Protocols:** Union operations aid in modeling communication protocols that support different valid message formats.
- **Automated Testing Tools:** Union of DFAs helps to create test cases that cater to varied input scenarios by merging assorted input patterns.

Limitations and Considerations

- **State Explosion Problem in Product Construction:** The intersection of two DFAs tends to cause the resulting combined automaton to have an equivalent state space size equal to the product of the original states, which causes it to grow exponentially.
- **Increased Requirements for Memory and Processing:** More computation is needed for larger DFAs, which can affect performance in real-world applications.

- **Alternative Solutions Using NFAs:** Non-deterministic Finite Automata (NFAs) provide a more space-efficient representation for union operations, which usually yields fewer states.
- **Regex Simplification Conversions:** Converting DFA to regular expression and then reversing could sometimes yield simpler automata for union but can complicate the conversion.
- **Trade-offs Between Determinism and Efficiency:** While DFAs guarantee deterministic processing, NFAs and regex-based techniques can provide more efficient union constructions but at the expense of non-determinism.
- **Minimization Techniques as Mitigation:** DFA minimization algorithms can be applied after union to minimize the number of states and increase efficiency.

2. Concatenation of Two DFAs ($L_1 \circ L_2$)

Concatenation means creating a new DFA that accepts words formed by taking a word from L_1 followed by a word from L_2 . Example:

Let,

- DFA 1 (L_1) accepts $L_1 = \{a, b\}$
- DFA 2 (L_2) accepts $L_2 = \{c, d\}$
- The concatenation DFA accepts $L_1 \circ L_2 = \{ac, ad, bc, bd\}$

Concatenation Process in DFA

In the context of Deterministic Finite Automata (DFA), concatenation refers to the process of combining two regular languages (or strings) together to form a new language.

- DFA is a model used to recognize patterns or decide if a string belongs to a particular language.
- Concatenation is simply taking two strings, one after the other. If you have a string A and a string B, the concatenation of these two strings results in AB (the first string followed by the second string).

How Concatenation Works in DFA

When you concatenate two regular languages (represented by DFAs), you essentially want to combine the DFAs that recognize each of the individual languages into one DFA that can recognize the concatenated language.

1. Initial Setup: Start with two separate DFAs, i.e. DFA1 and DFA2, where:

- DFA1 recognizes the first part of the string.
- DFA2 recognizes the second part of the string.

2. Modify the Final States: In the concatenation process, the final state of DFA1 is connected to the start state of DFA2. This means:

- Once DFA1 reaches its final state (after reading the first part of the string), the machine then transitions into DFA2 and starts reading the second part of the string.

3. Accepting States: The new DFA should accept the concatenated string if:

- The string is accepted by DFA1 (after processing the first part).

- Then, DFA2 should accept the second part of the string.

4. Final Automaton: After these adjustments, the new DFA recognizes the concatenation of both languages and can accept any string that is a valid combination of the two languages.

Example: Designing a DFA for the set of strings over {a, b} such that the string of the language starts with "a" and ends with "b". There two desired language will be formed:

$L1 = \{a, aab, aabab, \dots\}$

$L2 = \{b, bbab, bbabab, \dots\}$

1. Designing a DFA for Strings That Start with 'a' and End with 'b'

- The string starts with a
- The string ends with b

Examples of valid strings:

- ab
- aab
- aabab
- aaabbb

Examples of invalid strings:

- baba (does not start with a)
- aaba (ends with a)
- b (starts with b)

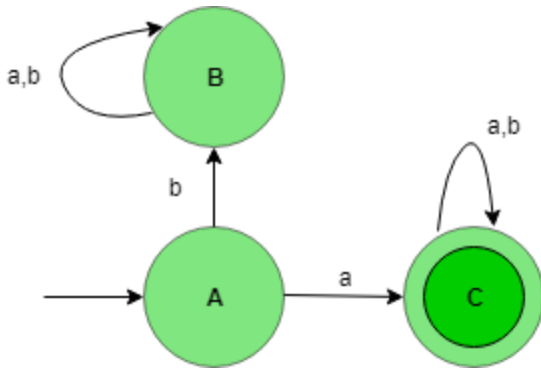
2. Defining Sub-Languages L1 and L2

2.1 Language L1: Strings that start with a

This DFA accepts any string that begins with the letter a. As soon as it sees the first character, if it's not a, it moves to a dead state (where no accepted string is possible anymore). After seeing an initial a, it can accept any combination of as and bs.

Examples: a, ab, aab, aaba, aaabb

Rejected: b, ba, bb

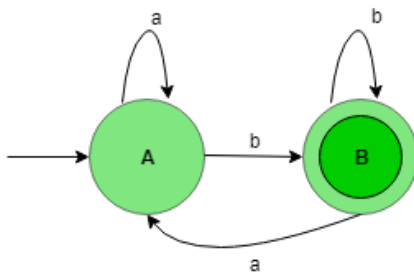


2.2 Language L2: Strings that end with b

This DFA accepts any string as long as its last character is b. It keeps track of the last symbol. If the last input symbol was b, the string is accepted. If the string ends in a, it's rejected.

Examples: b, ab, aab, bab

Rejected: a, aa, aba



3. Concatenation L1.L2

Now, to create the final DFA, we combine the logic of L1 and L2. We want a string that:

- Starts with a (L1)
- Ends with b (L2)

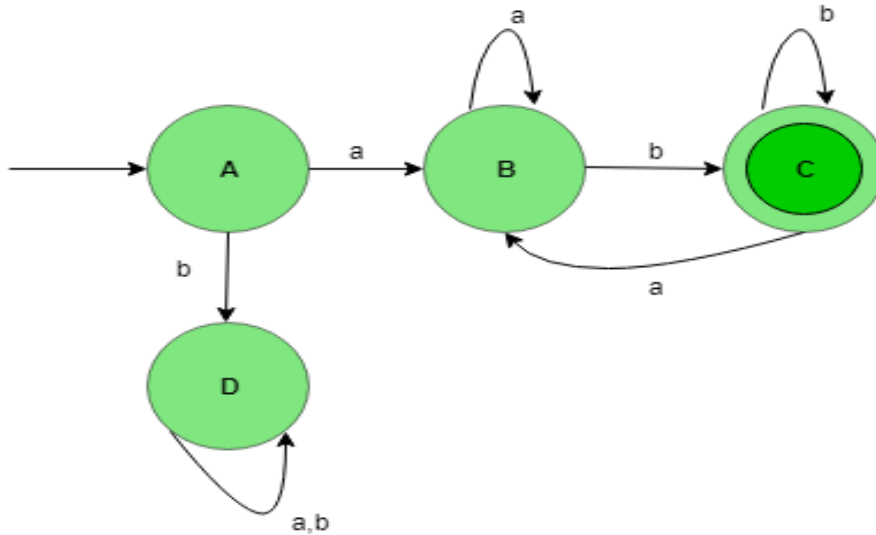
This means the DFA must first verify that the first letter is a, and after that, it must watch for the last letter to be b. Both conditions must be satisfied.

Examples of accepted strings:

- ab (starts with a, ends with b)
- aab
- aaabb
- aabab

Examples of rejected strings:

- aaba (ends with a)
- b (starts with b)
- baab (starts with b)
- a (ends with a)



3. Reversal of a DFA (L^R)

A new DFA that accepts the reversed versions of words accepted by the original DFA. Example:

- Let DFA accept $L = \{ab, abc, ba\}$
- Then, reversed DFA accepts $L^R = \{ba, cba, ab\}$

Pre-requisite: Designing Finite Automata from Regular Expression

A regular expression is a simple way to describe patterns in text, like “any number of a's” (a^*) or “either a or b” ($a|b$). A finite automaton is a basic machine that reads a string one letter at a time and decides if it matches a given pattern. Converting a regular expression to a finite automaton means turning the pattern into a machine that can automatically check if a string follows the pattern, which is useful in tools like text editors, programming languages, and compilers. In this article, we will see some popular regular expressions and how we can convert them to finite automata.

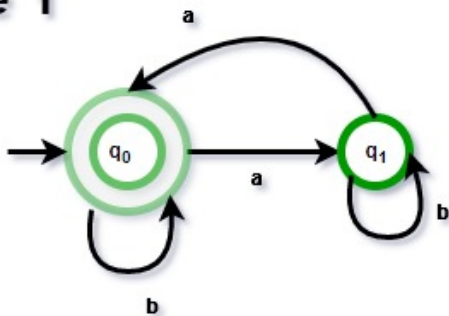
Even number of a's

The regular expression for even number of a's is

$(b|ab^*ab^*)^*$

We can construct a finite automata as shown in figure below.

Figure 1



The above automata will accept all strings which have an even number of a's. For zero a's, it will be in q_0 which is the final state. For one 'a', it will go from q_0 to q_1 and the string will not be accepted. For two a's at any positions, it will go from q_0 to q_1 for 1st 'a' and q_1 to q_0 for second 'a'. So, it will accept all strings with an even number of a's.

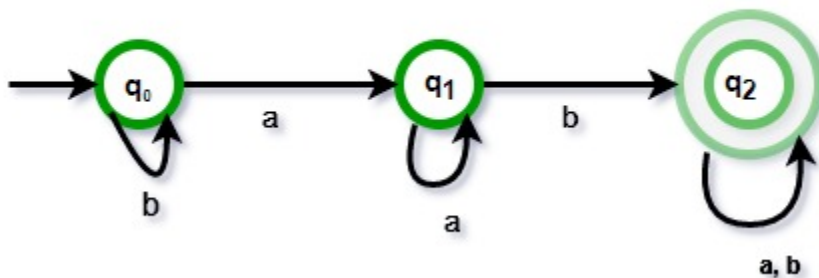
String with 'ab' as substring

The regular expression for strings with 'ab' as substring is

$$(ab)^*ab(ab)^*$$

We can construct finite automata as shown in figure below.

Figure 2



The above automata will accept all strings which have 'ab' as substring. The automata will remain in initial state q_0 for b's. It will move to q_1 after reading 'a' and remain in the same state for all 'a' afterwards. Then it will move to q_2 if 'b' is read. That means, the string has read 'ab' as a substring if it reaches q_2 .

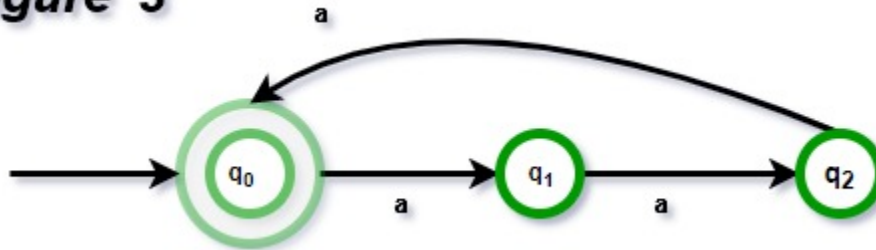
String with count of 'a' divisible by 3

The regular expression for strings with count of a divisible by 3 is

$$\{a^{3n} \mid n \geq 0\}$$

We can construct automata as shown in Figure 3.

Figure 3



The above automata will accept all string of form a^{3n} . The automata will remain in initial state q_0 for ? and it will be accepted. For string 'aaa', it will move from q_0 to q_1 then q_1 to q_2 and then q_2 to q_0 . For every set of three a's, it will come to q_0 , hence accepted. Otherwise, it will be in q_1 or q_2 , hence rejected.

Note: If we want to design a finite automata with number of a's as $3n+1$, the same automata can be used with the final state as q_1 instead of q_0 . If we want to design a finite automata with language $\{a^{kn} \mid n \geq 0\}$, k states are required. We have used $k = 3$ in our example.

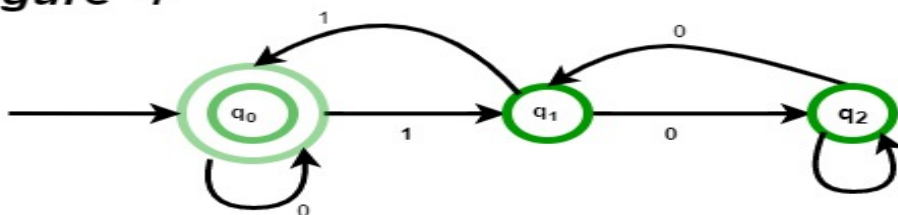
Binary numbers divisible by 3

The regular expression for binary numbers which are divisible by three is

$$(01(01^*0)^*1)^*$$

The examples of binary numbers divisible by 3 are 0, 011, 110, 1001, 1100, 1111, 10010 etc. The DFA corresponding to the binary number divisible by 3 can be shown in Figure 4.

Figure 4

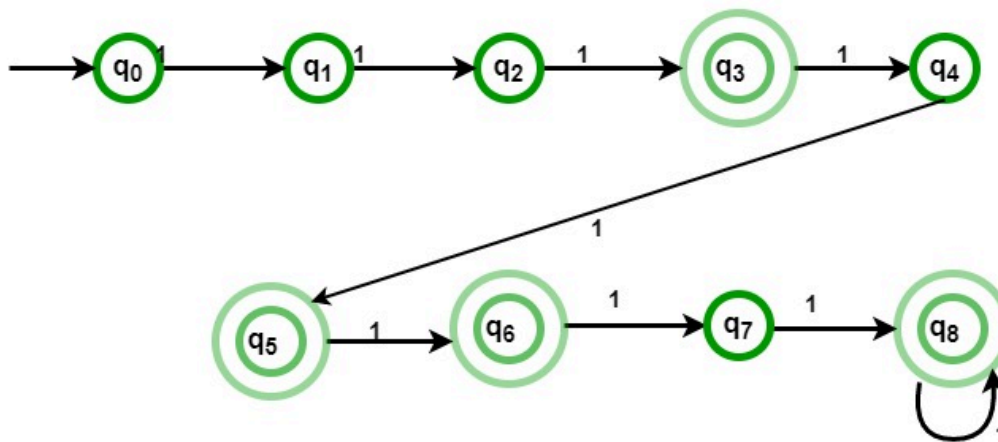


The above automata will accept all binary numbers divisible by 3. For 1001, the automata will go from q_0 to q_1 , then q_1 to q_2 , then q_2 to q_1 and finally q_2 to q_0 , hence accepted. For 0111, the automata will go from q_0 to q_0 , then q_0 to q_1 , then q_1 to q_0 and finally q_0 to q_1 , hence rejected.

String with regular expression $(111 + 11111)^*$

The string accepted using this regular expression will have 3, 5, 6(111 twice), 8 (11111 once and 111 once), 9 (111 thrice), 10 (11111 twice) and all other counts of 1 afterwards. The DFA corresponding to a given regular expression is given in Figure 5.

Figure 5

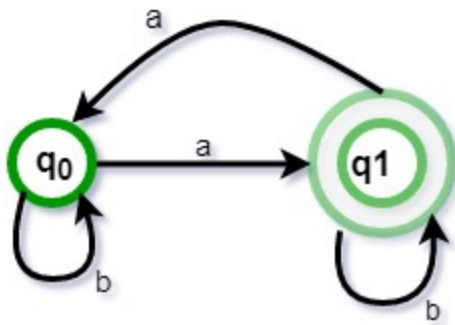


What will be the minimum number of states for strings with an odd number of a's?

Solution : The regular expression for odd number of a is

$b^*ab^*(ab^*ab^*)^*$

Corresponding automata are given here and the minimum number of states are 2.



Reversing Deterministic Finite Automata

Prerequisite – [Designing finite automata](#)

Reversal: We define the reversed language L^R of L to be the language $L^R = \{w^R \mid w \in L\}$, where $w^R := a_n a_{n-1} \dots a_1 a_0$ for $w = a_0 a_1 \dots a_{n-1} a_n$

Steps to Reversal:

1. Draw the states as it is.
2. Add a new single accepting state
3. Make the accepting state the initial state and initial state the accepting state
4. Invert the transition function into a transition relation
5. Remove any states that are no longer necessary

Note :

Reversing a deterministic finite automaton results in a non-deterministic finite automaton. This can then be converted to a deterministic finite automaton by the conversion algorithm.

The reversal process goes like this:



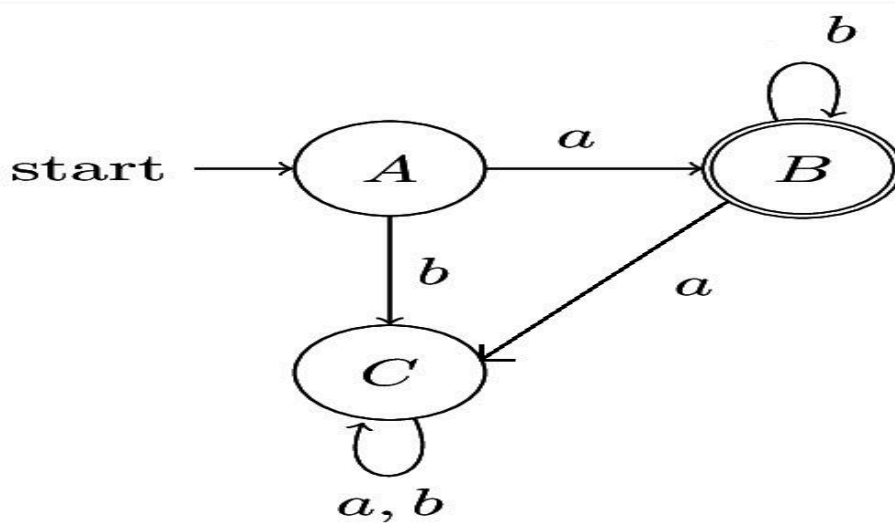
Let's understand the Reversal process in Deterministic Finite Automata (DFA) with the help of the example below.

Example 1:

We propose a deterministic finite automaton for the language of words beginning with the symbol 'a':

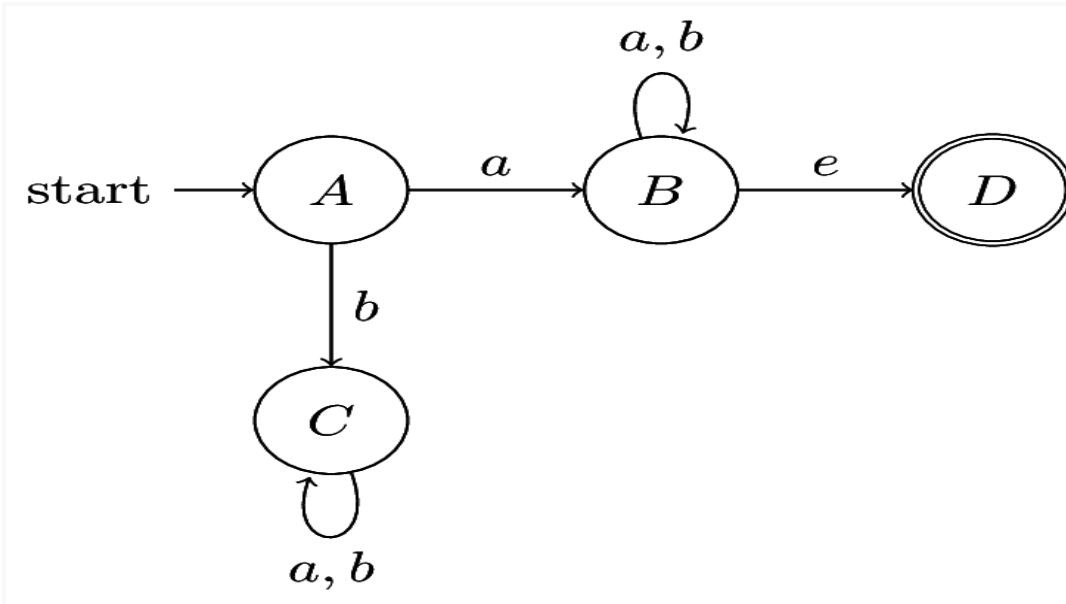
$$L_1 = \{w \in \{a, b\}^* \mid w = ab^n \text{ for some } n \geq 0\}$$

We can construct the following state transition diagram:

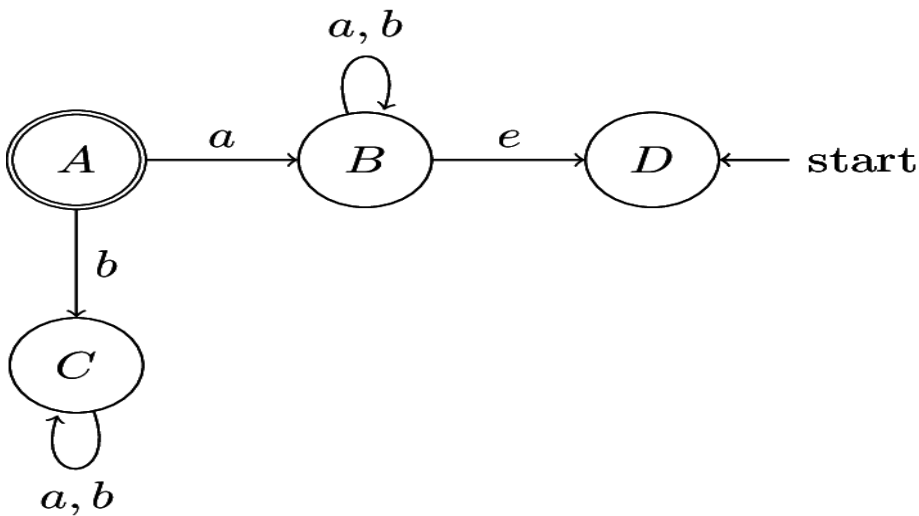


We now want to construct the finite automaton that will accept the reverse of this language, L_2 .

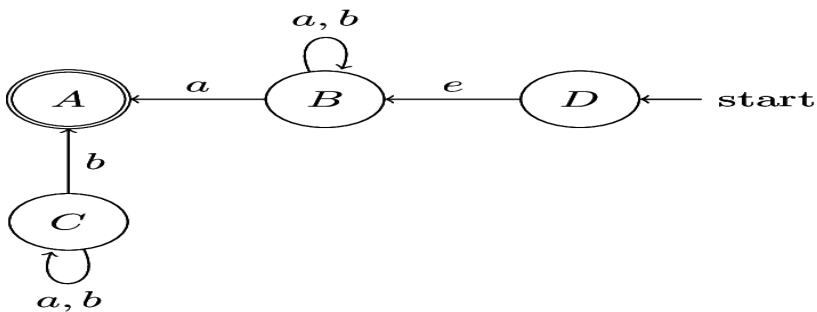
First, we add a new single accepting state. This is because eventually we will need to swap the accepting and initial states, and there can only be a single initial state.



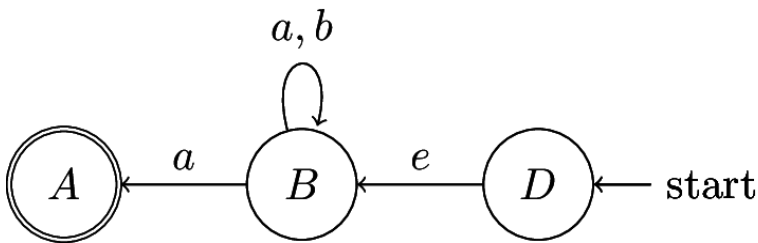
Then, switch the initial and accepting states:



Invert the transition function into a transition relation:



Whilst this is a valid solution, the node marked as C is dangling since it has no incident edges. Therefore it can be removed to give a clean, final result:



This resulting finite automaton is a non-deterministic finite automaton. Therefore the resulting reversed language is regular.

Explanation:

1. Same states (A, B, C) are drawn as present in an original state diagram
2. Since A is the initial state, make it the final state.
3. Since B is the final state, make it the initial state.
4. Reversing the edges, since previously edges point A to B and A to C. Change the direction, now edges point toward B to A and C to A.
5. Assign the values the same as the original.
6. Draw the loop as in the original state diagram
7. Since there is no incident edge on state C so, we can reduce this state C.
8. Since there is no transition for 'a' and 'b' in state A. Therefore the resultant FA is NFA.

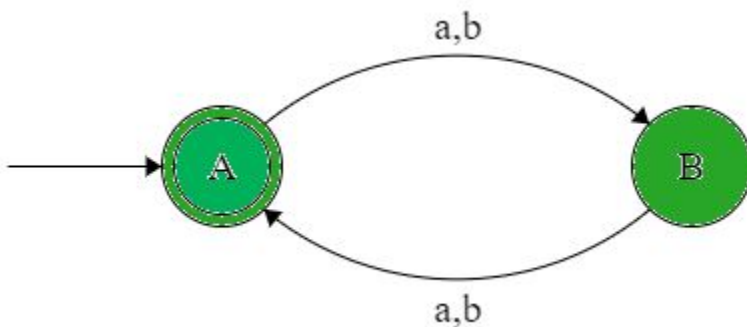
Example-2:

Designing a DFA for the set of strings over $\{a, b\}$ such that the string of the language contains an even number of 'a'. The desired language will be formed:

$L1 = \{\epsilon, aa, aab, aba, aab, aababa, \dots\}$

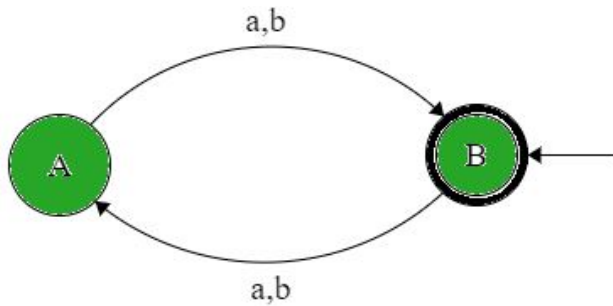
In $L1$, each string has an even number of elements a.

State Transition Diagram for the language $L1$:



This DFA accepts all the strings containing an even number of elements a. Here, State A is the initial state, and state A is the final state. Now, reverse the $L1$ language which gives the final result of the language $L2$.

State Transition Diagram of $L2$ (reverse of $L1$):



Thus as we see that L_2 has been obtained through the reversal process and this final DFA accepts all the strings containing an even number of elements a which is the same as the original language.

$L_2 = \{\epsilon, aa, aab, aba, aab, aababa, \dots\}$

Explanation:

1. Same states (A, B) are drawn as present in an original state diagram.
2. Since A is the initial state, make it the final state.
3. Since A is the final state, make it the initial state.
4. Reversing the edges, since previously edges point A to B and so on. Change the direction, now edge points toward B to A and so on.
5. Assign the values the same as the original.
6. Since there is a transition for 'a' and 'b' in states A and B. Therefore resultant FA is DFA.
7. Since the State diagram after and before the reversal is the same. Therefore $L_1 = L_2$

4. Complementation of a DFA (L^c)

A new DFA that accepts all words not accepted by the original DFA. Example:

- Let DFA accept $L = \{\text{all strings containing 'a'}\}$
- Then, the complement DFA accepts $L^c = \{\text{all strings that do not contain 'a'}\}$

Complementation process in DFA

Suppose we have a DFA that is defined by $(Q, \Sigma, \delta, q_0, F)$ and it accepts the language L_1 . Then, the DFA which accepts the language L_2 where $L_2 = \overline{L_1}$, will be defined as below:

$$(Q, \Sigma, \delta, q_0, Q-F)$$

The complement of a DFA can be obtained by **making the non-final states as final states and vice-versa**. The language accepted by the complemented DFA L_2 is the complement of the language L_1 .

Example-1:

L_1 : set of all strings over $\{a, b\}$ of even length

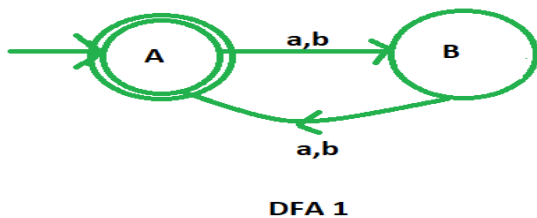
$$L_1 = \{\epsilon, ab, aa, abaa, aaba, \dots\}$$

L_2 : set of all strings over $\{a, b\}$ of odd length

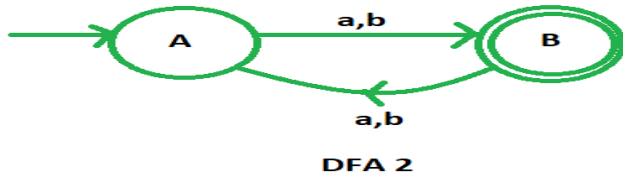
$$L_2 = \{a, b, aab, aaa, bba, bbb, \dots\}$$

Here, we can see that $L_2 = \overline{L_1}$

Lets first draw the DFA for L_1 that accepts the strings of even length.



Now, for designing the DFA for $L2$, we just need to complement the above DFA. We will change the non-final states as final states and the final states as non-final states.



This is our required complemented DFA.

Example-2:

$L1$: set of all strings over $\{a, b\}$ starting with 'a'.

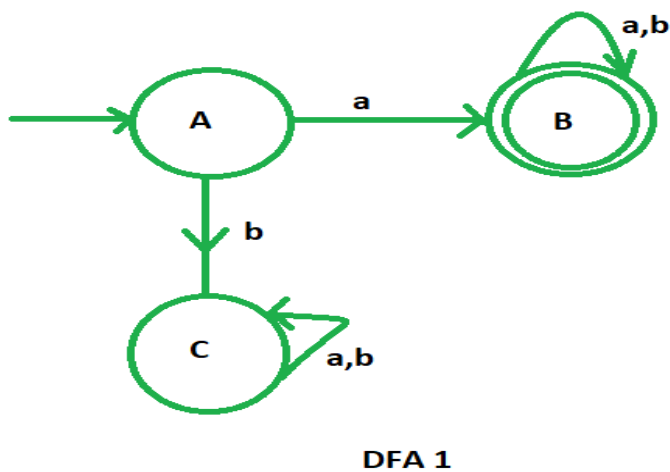
$L1 = \{a, ab, aa, aba, aaa, aab, ..\}$

$L2$: set of all strings over $\{a, b\}$ not starting with 'a'.

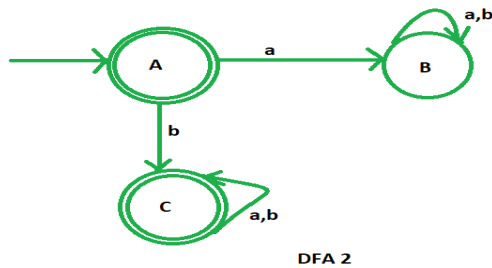
$L2 = \{\epsilon, b, ba, bb, bab, baa, bba, ...\}$

Here, we can see that $L2 = \overline{L1}$

Lets first draw the DFA for $L1$ that accepts the set of all strings over $\{a, b\}$ starting with 'a'



Now, for designing the DFA for L_2 , we just need to complement the above DFA. We will change the non-final states as final states and the final states as non-final states.



This is our required complemented DFA that accepts the strings that are not starting with 'a'.

Note: Regular languages are closed under complement (i.e Complement of regular language will also be regular).

5. Intersection of Two DFAs ($L_1 \cap L_2$)

A new DFA that accepts only words that are in both original DFAs. Example:
Let:

- DFA 1 (L_1) accepts $L_1 = \{a, ab, bc\}$
- DFA 2 (L_2) accepts $L_2 = \{ab, bc, cd\}$
- The intersection DFA accepts $L_1 \cap L_2 = \{ab, bc\}$

Intersection process of two DFAs

Let's understand the intersection of two DFA with an example.

Designing a DFA for the set of strings over $\{0, 1\}$ such that it ends with 01 and has an even number of 1's. There two desired language will be formed:

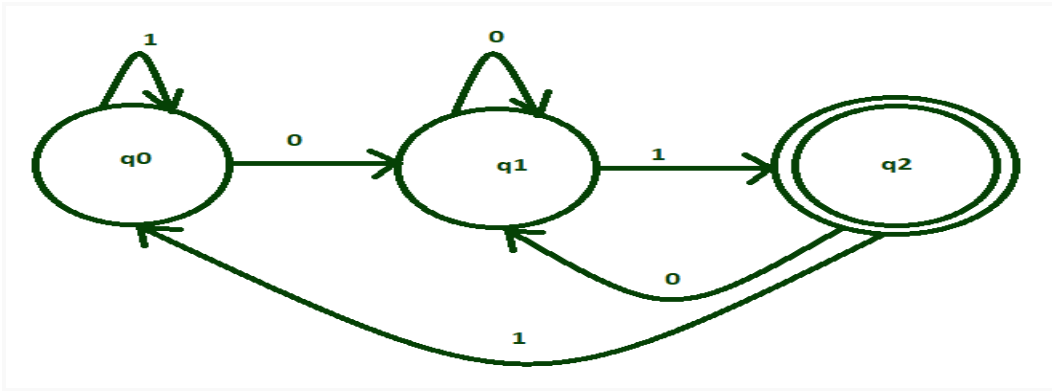
$L_1 = \{01, 001, 101, 0101, 1001, 1101, \dots\}$

$L_2 = \{11, 011, 101, 110, 0011, 1100, \dots\}$

$L = L_1 \text{ and } L_2 = L_1 \cap L_2$

State Transition Diagram for the language L_1 :

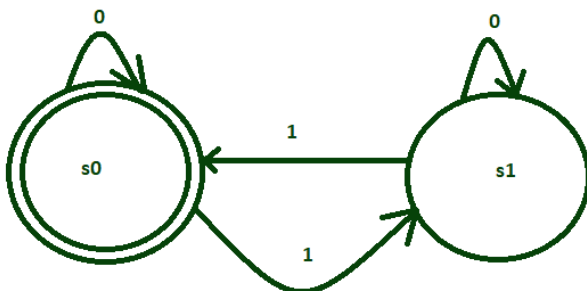
This is a DFA for language L_1



It accepts all the strings that accept 01 at the end.

State Transition Diagram for the language L2 :

This is a DFA for language L2



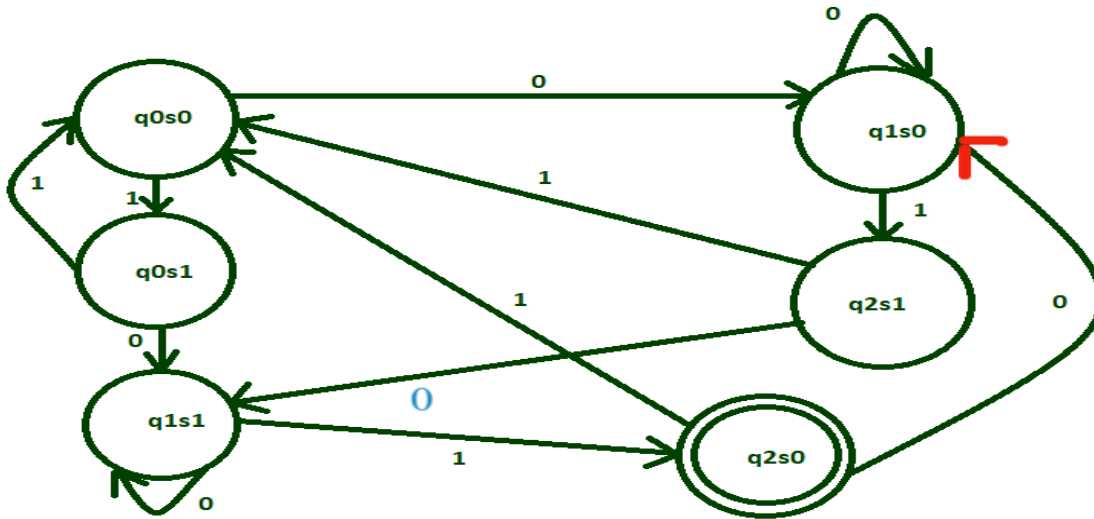
It accepts all the strings that accept an even number of 1's.

State Transition Diagram of $L1 \cap L2$:

Intersection of L1 and L2 can be explained by language that a string over $\{0, 1\}$ accepts such that it ends with 01 and has even number of 1's.

$$L = L1 \cap L2$$

$$= \{1001, 0101, 01001, 10001, \dots\}$$



Thus as we see that L_1 and L_2 have been combined through an intersection process and this final DFA accepts all the languages that have an even number of 1's and is ending with 01.

6. Difference of Two DFAs ($L_1 - L_2$)

Accepts strings in A but **not** in B

Let :

- DFA A accepts strings containing at least one '0'.
- DFA B accepts strings ending with '1'.
- Difference DFA $L_1 - L_2$ accepts strings that have at least one '0' and do not end with '1'.

Conversion from NFA to DFA

An NFA can have zero, one or more than one move from a given state on a given input symbol. An NFA can also have NULL moves (moves without input symbol). On the other hand, DFA has one and only one move from a given state on a given input symbol.

Steps for converting NFA to DFA:

Step 1: Convert the given NFA to its equivalent transition table

To convert the NFA to its equivalent transition table, we need to list all the states, input symbols, and the transition rules. The transition rules are represented in the form of a matrix, where the rows represent the current state, the columns represent the input symbol, and the cells represent the next state.

Step 2: Create the DFA's start state

The DFA's start state is the set of all possible starting states in the NFA. This set is called the "epsilon closure"

of the NFA's start state. The epsilon closure is the set of all states that can be reached from the start state by following epsilon transitions.

Step 3: Create the DFA's transition table

The DFA's transition table is similar to the NFA's transition table, but instead of individual states, the rows and columns represent sets of states. For each input symbol, the corresponding cell in the transition table contains the epsilon closure of the set of states obtained by following the transition rules in the NFA's transition table.

Step 4: Create the DFA's final states

The DFA's final states are the sets of states that contain at least one final state from the NFA.

Step 5: Simplify the DFA

The DFA obtained in the previous steps may contain unnecessary states and transitions. To simplify the DFA, we can use the following techniques:

- Remove unreachable states: States that cannot be reached from the start state can be removed from the DFA.
- Remove dead states: States that cannot lead to a final state can be removed from the DFA.
- Merge equivalent states: States that have the same transition rules for all input symbols can be merged into a single state.

Step 6: Repeat steps 3-5 until no further simplification is possible

After simplifying the DFA, we repeat steps 3-5 until no further simplification is possible. The final DFA obtained is the minimized DFA equivalent to the given NFA.

Example: Consider the following NFA shown in Figure 1.

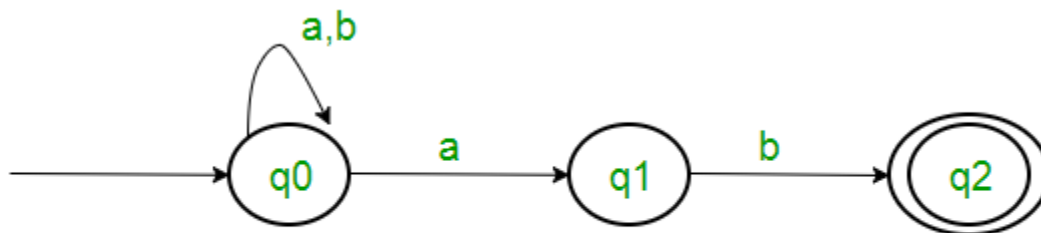


Figure 1

Following are the various parameters for NFA. $Q = \{ q_0, q_1, q_2 \}$, $\Sigma = (a, b)$, $F = \{ q_2 \}$ (Transition Function of NFA)

State	a	b
q0	q0,q1	q0
q1		q2
q2		

State	a	b
q0	{q0,q1}	q0

- Start with DFA-state $\{q_0\}$.
- On **a**: $\{q_0\} \rightarrow \{q_0, q_1\}$; on **b**: $\{q_0\} \rightarrow \{q_0\}$. So we add $\{q_0, q_1\}$ to DFA state set. GeeksforGeeks +1
- Now for DFA-state $\{q_0, q_1\}$:
 - On **a**: union of transitions from q_0 and q_1 on **a** $\rightarrow \{q_0, q_1\}$
 - On **b**: union of transitions $\rightarrow \{q_0, q_2\}$. Add $\{q_0, q_2\}$. GeeksforGeeks
- Next for $\{q_0, q_2\}$: compute for **a** and **b**. Suppose transitions go to previously known sets $\rightarrow$ no new sets. End construction. GeeksforGeeks +1
- Among the DFA states, any that include q_2 (the NFA final state) become accepting. Here $\{q_0, q_2\}$ is accepting. GeeksforGeeks

Thus we obtain a DFA whose states are: $\{q_0\}, \{q_0, q_1\}, \{q_0, q_2\}$; start state $\{q_0\}$; accept state $\{q_0, q_2\}$; with deterministic transitions defined for **a** and **b**. GeeksforGeeks +1

State	a	B
q0	{q0,q1}	q0
{q0,q1}	{q0,q1}	{q0,q2}

State	a	B
q0	{q0,q1}	q0
{q0,q1}	{q0,q1}	{q0,q2}
{q0,q2}	{q0,q1}	q0

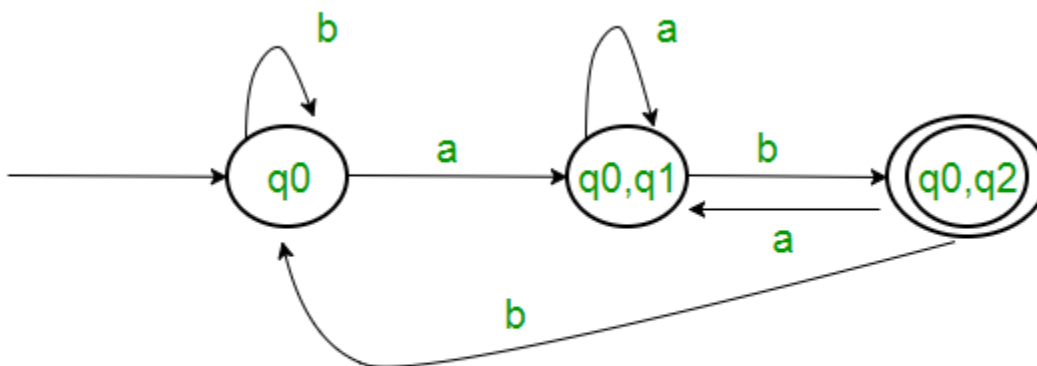
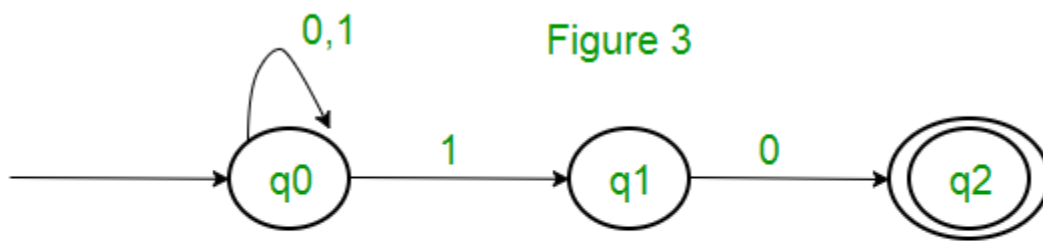


Figure 2

Note : Sometimes, it is not easy to convert regular expressions to DFA. First you can convert regular expressions to NFA and then NFA to DFA.

Question: The number of states in the minimal deterministic finite automaton corresponding to the regular expression $(0 + 1)^* (10)$ is _____?_____.

Solution : First, we will make an NFA for the above expression. To make an NFA for $(0 + 1)^*$, NFA will be in the same state q_0 on input symbol 0 or 1. Then for concatenation, we will add two moves (q_0 to q_1 for 1 and q_1 to q_2 for 0) as shown in Figure 3.



Using above algorithm, we can convert NFA to DFA as shown in Figure 4

State	0	1
q0	q0	q0,q1
q0,q1	q0,q2	q0,q1
q0,q2	q0	q0,q1

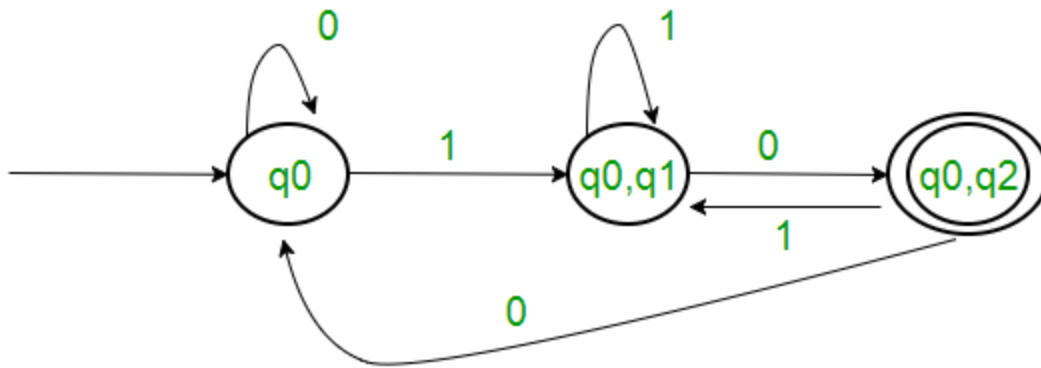
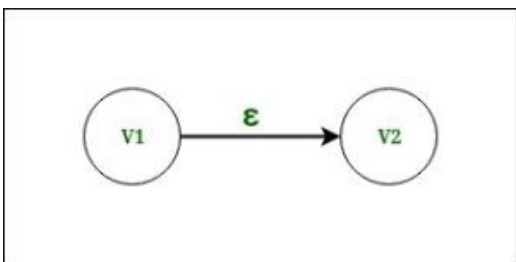


Figure 4

Conversion of Epsilon-NFA to NFA

An NFA (Non-deterministic Finite Automaton) is a theoretical model used to recognize patterns and determine whether a string belongs to a specific language. An Epsilon-NFA (ϵ -NFA) is a special type of NFA where transitions can happen without reading any input symbol, using epsilon (ϵ) transitions.

- An NFA allows multiple possible transitions from a given state based on the input symbol.
- The key difference between them is that Epsilon-NFAs have epsilon transitions, while regular NFAs only transition on specific input symbols.
- Converting an Epsilon-NFA to an NFA eliminates the epsilon transitions, allowing us to apply standard NFA algorithms for language recognition.



Epsilon-NFA to NFA Conversion Steps

Step 1: Find the Epsilon Closure

- For each state in the Epsilon-NFA, find its epsilon closure.

- This means figuring out all the states that can be reached by only using epsilon transitions (and include the state itself).
- This helps in tracking which states can be reached without reading any input symbol.

Step 2: Create New States for the NFA

- Each state in the new NFA corresponds to a set of states you found in the epsilon closures.
- The starting state in the NFA will be the epsilon closure of the initial state of the Epsilon-NFA.

Step 3: Define the Transitions

- For each state in the NFA (which represents a group of states from the epsilon closures), look at what happens when you read each input symbol.
- For an input symbol a , check which states you can reach from any state in the current group, and then find the epsilon closure of those states.
- This way, you're considering both direct transitions and the ones that happen because of epsilon moves.

Step 4: Set Accepting States

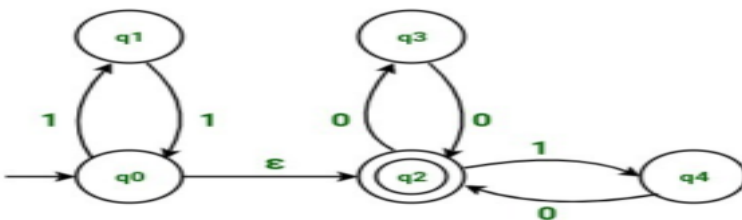
- Any state in the NFA that includes at least one accepting state from the Epsilon-NFA becomes an accepting state.
- This ensures that the NFA recognizes all the same strings as the Epsilon-NFA.

Step 5: Keep Going Until Done

- Keep processing each new state and its transitions until no more new states are created.
- This guarantees that every possible state is covered, and there are no epsilon transitions left.

Example of Epsilon-NFA to NFA Conversion

Convert the following ϵ -NFA to NFA. The ϵ -NFA has states q_0, q_1, q_2, q_3, q_4 where q_0 is the initial state and q_2 final state.



Solution: In the above example, we have 5 states named as q0, q1, q2, q3 and q4. Initially, we have q0 as the start state and q2 as the final state. We have q1, q3 and q4 as intermediate states.

According to ϵ -NFA the states follow following transitions:

- State q0 when,
 - **Input 0:** No transition
 - **Input 1:** Moves to q1
 - **Input ϵ :** Moves to q2
- State q1 when,
 - **Input 0:** No transition
 - **Input 1:** Moves to q0
 - **Input ϵ :** No transition
- State q2 when,
 - **Input 0:** Moves to q3
 - **Input 1:** Moves to q4
 - **Input ϵ :** No transition
- State q3 when,
 - **Input 0:** Moves to q2
 - **Input 1:** No transition
 - **Input ϵ :** No transition
- State q4 when,
 - **Input 0:** Moves to q2
 - **Input 1:** No transition
 - **Input ϵ :** No transition

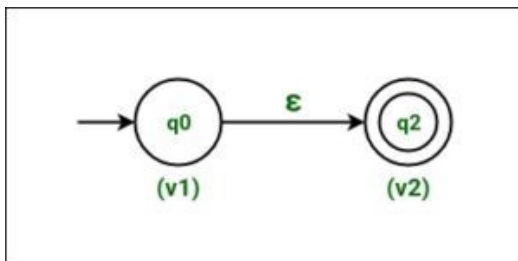
Transition table for the state transitions:

States/Input	Input 0	Input 1	Input epsilon
q0	-	q1	q2
q1	-	q0	-

q2	q3	q4	-
q3	q2	-	-
q4	q2	-	-

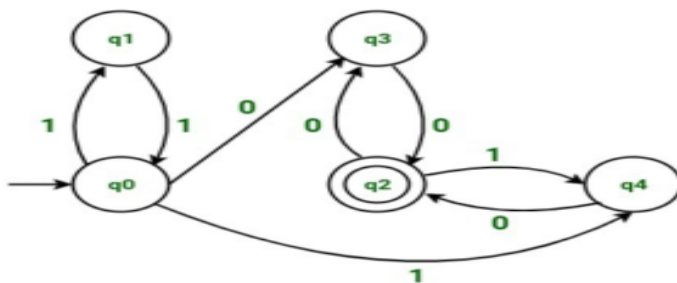
We can see that we have an epsilon move from state q_0 to state q_2 , which is to be removed. To remove the epsilon move from state q_0 to state q_1 , we will follow the steps mentioned below.

Step 1: Consider the epsilon move from state q_0 to state q_2 . Consider the state q_0 as vertex v_1 and state q_2 as vertex v_2 .

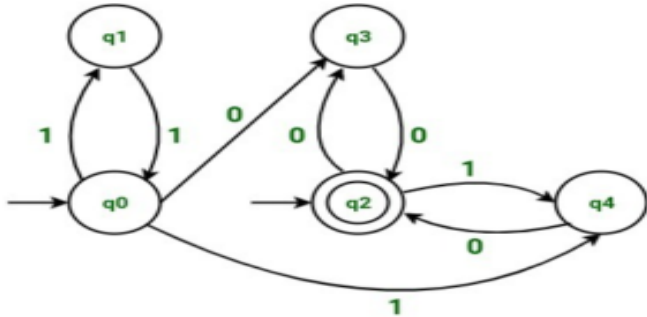


Step 2: Now find all the moves that start from vertex v_2 (i.e. state q_2). After finding the moves, duplicate all the moves that start from vertex v_2 (i.e. state q_2) with the same input to start from vertex v_1 (i.e. state q_0) and remove the epsilon move from vertex v_1 (i.e. state q_0) to vertex v_2 (i.e. state q_2). Since state q_2 on getting input 0 goes to state q_3 . Hence on duplicating the move, we will have state q_0 on getting input 0 also to go to state q_3 .

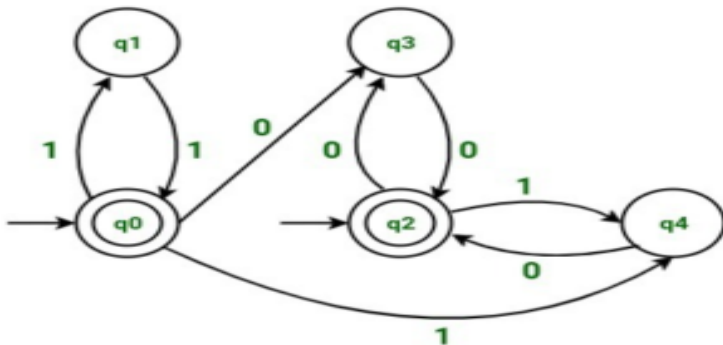
Similarly state q_2 on getting input 1 goes to state q_4 . Hence on duplicating the move, we will have state q_0 on getting input 1 also to go to state q_4 . So, NFA after duplicating the moves is:



Step 3: Since vertex v1 (i.e. state q0) is a start state. Hence we will also make vertex v2 (i.e. state q2) as a start state. Note that state q2 will also remain as a final state as we had initially. NFA after making state q2 also as a start state is:



Step 4: Since vertex v2 (i.e. state q2) is a final state. Hence we will also make vertex v1 (i.e. state q0) as a final state. Note that state q0 will also remain as a start state as we had initially. After making state q0 also as a final state, the resulting NFA is:



The transition table for the above resulting NFA is:

States/Input	Input 0	Input 1
q0	q3	q1,q4

q1	-	q0
q2	q3	q4
q3	q2	-
q4	q2	-

Equivalence Of F.S.A (Finite State Automata)

An Automaton is a machine that has a finite number of states. Any Two Automaton is said to be equivalent if both accept exactly the same set of input strings.

Two Automaton are equivalent if they satisfy the following conditions :

- 1. The initial and final states of both the automata must be same.*
- 2. Every pair of states chosen is from a different automaton only.*
- 3. While combining the states with the input alphabets, the pair results must be either both final states or intermediate states.(i.e both should lie either in the final state or in the non-final state).*
- 4. If the resultant pair has different types of states, then it will be non-equivalent. (i.e. One lies in the final state and the other lies in the intermediate state).*

Example 1 -

Consider Two Different Automaton shown below in Figure 1.

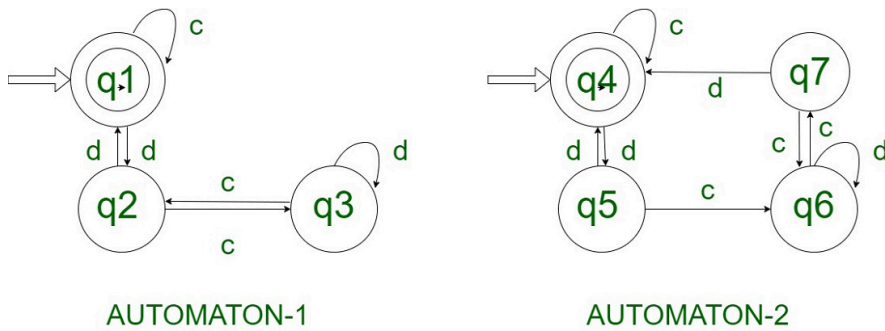


FIGURE -1

Solution -

Step 1 - Since the initial and final states of both the automaton are the same, so it verifies.

Step 2 - Check for each state by making a table of states with respect to input alphabets.

States	c	d
(q1,q4)	(q1,q4)=>(F.S,F.S)	(q2,q5)=>(I.S,I.S)
(q2,q5)	(q3,q6)=>(I.S,I.S)	(q1,q4)=>(F.S,F.S)
(q3,q6)	(q2,q7)=>(I.S,I.S)	(q3,q6)=>(I.S,I.S)
(q2,q7)	(q3,q6)=>(I.S,I.S)	(q1,q4)=>(F.S,F.S)

Here ,

F.S represents -> Final State.

I.S represents -> Intermediate State (Non-Final State) .

Step 3 - For every pair of states, the resultant states lie either in F.S or in I.S as a combination.

Conclusion - Both the Automaton are equivalent.

Note : If the resultant pair of states has a different combination of states(i.e. either (F.S, I.S) or (I.S, F.S), then both the automata are said to be non-equivalent.

Example 2 -

Consider Two Different Automaton shown below in Figure 2.

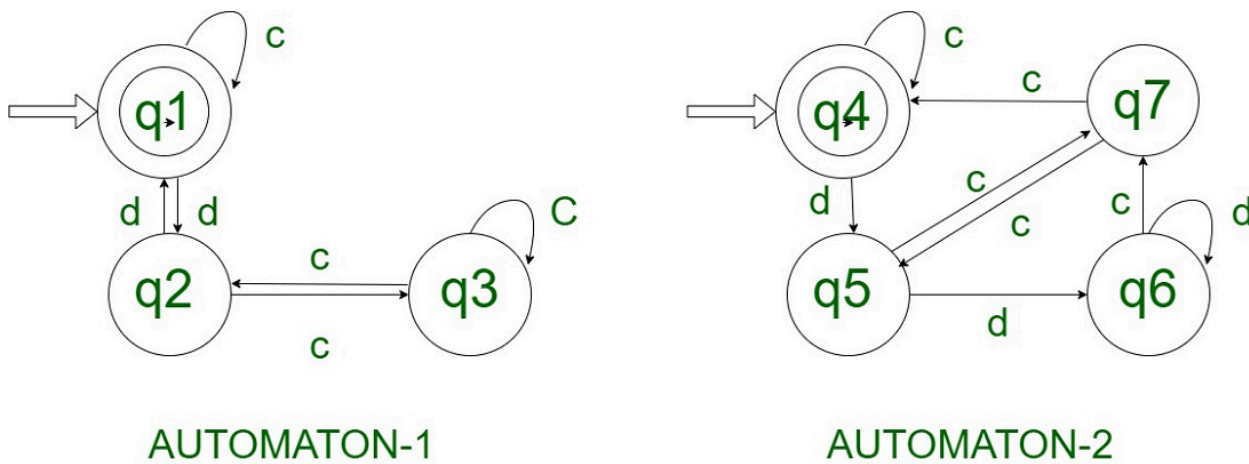


FIGURE -2

Solution -

Step 1 - Since the initial and final states of both the automaton are the same so it verifies.

Step 2 - Check for each state by making a table of states with respect to input alphabets.

States	c	d
(q1,q4)	(q1,q4)=>(F.S,F.S)	(q2,q5)=>(I.S,I.S)
(q2,q5)	(q3,q7)=>(I.S,I.S)	(q1,q6)=>(F.S,I.S) This is not a valid pair

Here ,

F.S represents -> Final State.

I.S represents -> Intermediate State (Non-Final State) .

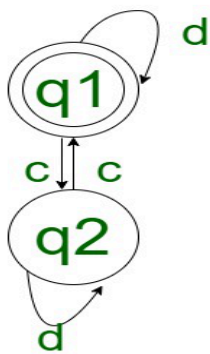
Step 3 - For the first pair of states, the resultant states lie in F.S as a combination

Step 4 - But for the pair (q2,q5) when operated over input alphabets they lie in different states. (i.e. one in F.S and the other in I.S).

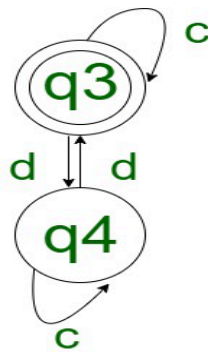
Conclusion - The Given Automata are not equivalent.

Example 3 -

Consider Two Different Automaton shown below in Figure 3.



AUTOMATON-1



AUTOMATON-2

FIGURE -3

Solution -

Step 1 - Since the initial and final states of both the automaton are the same so it verifies.

Step 2 - Check for each state by making a table of states with respect to input alphabets.

States	C	d
(q1,q3)	$(q2,q3) \longrightarrow (I.S, F.S)$ (This is not a valid pair)	$(q1,q4) \longrightarrow (F.S, I.S)$ (This is not a valid pair)

Here,

F.S represents -> Final State.

I.S represents -> Intermediate State (Non-Final State) .

Step 3 - For the pair (q1,q3), when operated over input alphabets they lie in different states. (i.e. one in F.S and the other in I.S).

Conclusion - The Given Automata are not equivalent.

Example 4 -

Consider Two Different Automaton shown below in Figure 4.

Solution -

Step 1 - Since the initial and final states of both the automaton are the same, so it verifies.

Step 2 - Check for each state by making a table of states with respect to input alphabets.

States	c	d
(q1,q4)	$(q2,q5) \Rightarrow (I.S, I.S)$	$(q1,q4) \Rightarrow (F.S, F.S)$
(q2,q5)	$(q3,q5) \Rightarrow (I.S, I.S)$	$(q2,q6) \Rightarrow (I.S, I.S)$
(q3,q5)	$(q3,q5) \Rightarrow (I.S, I.S)$	$(q2,q6) \Rightarrow (I.S, I.S)$
(q2,q6)	$(q3,q5) \Rightarrow (I.S, I.S)$	$(q2,q6) \Rightarrow (I.S, I.S)$

Here,

F.S represents -> Final State.

I.S represents -> Intermediate State (Non-Final State) .

Step 3 - For every pair of states, the resultant states lie either in F.S or in I.S as a combination.

Conclusion - Both the Automaton are equivalent.